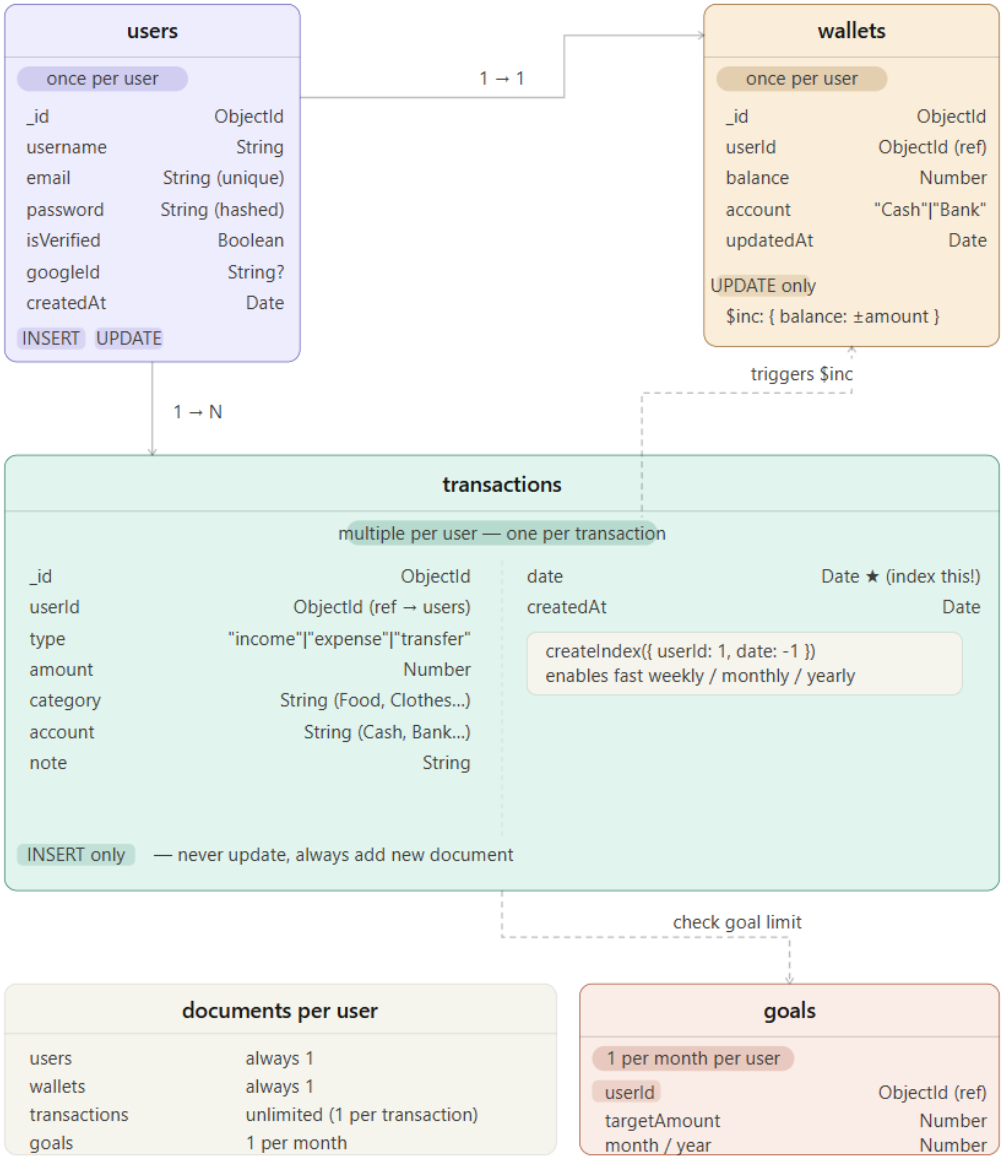
Expense Tracker App

Backend — Complete Discussion Notes

MongoDB Schema • Date Queries • API Routes • Key Rules



1. Overview — What We Decided

After full discussion, we finalized exactly 4 MongoDB collections for this app. Nothing more is needed.

- users — stores signup / login info for every user
- wallets — stores current balance only (always 1 per user)
- transactions — stores every expense and income (unlimited per user)
- goals — stores monthly saving target (1 per month per user)

> *transactions collection does ALL the heavy lifting — weekly, monthly, yearly data, pie chart, bar chart all come from here.*

2. MongoDB Collections — Full Detail

A. users

Purpose: Handle signup, login, and profile for every user.

Collection: users

Documents per user: **Always 1 — one account per user**

Operation: **INSERT on signup | UPDATE on profile change**

```
{ _id: ObjectId,
  username: String,
  email: String,        // unique — no duplicate emails
  password: String,     // bcrypt hashed — never plain text
  isVerified: Boolean,  // true after OTP verified
  googleId: String,     // only for Google OAuth login
  createdAt: Date }
```

> *Hash password with bcrypt before saving. Never store plain text.*
> *Use JWT for authentication. Store token in HTTP-only cookie.*
> *For Google login use OAuth 2.0 — googleId fills automatically on first login.*
> *OTP: generate 6-digit random number, send to email, verify, set isVerified = true.*

B. wallets

Purpose: Store the current balance of the user. Think of it like a scoreboard — just one number that keeps changing.

Collection: wallets

Documents per user: **Always 1 — one wallet per user forever**

Operation: **UPDATE only — never INSERT a new one after first time**

```
{ _id: ObjectId,
  userId: ObjectId,     // ref → users
  balance: Number,      // current balance (keeps updating)
  account: String,      // 'Cash' | 'Bank'
  updatedAt: Date }
```

How to update balance:

```
// When expense happens — subtract
db.wallets.updateOne(
  { userId: ObjectId('abc123') },
  { $inc: { balance: -500 } }   // -500 subtracts automatically
)

// When income happens — add
db.wallets.updateOne(
```

```
  { userId: ObjectId('abc123') },
  { $inc: { balance: +5000 } }  // +5000 adds automatically
)
```

> *$inc does the math automatically on the server. No need to fetch balance first and calculate manually.*
>
> *MongoDB finds the SAME single document every time and just updates the number. It does not care about history.*
>
> *wallets = piggy bank (one box, balance keeps changing). transactions = notebook (new line for every expense).*

C. transactions

Purpose: Store every single expense and income entry. This is the most important collection — all charts and reports come from here.

Collection: transactions

Documents per user: **Unlimited — every expense or income = 1 new document**

Operation: **INSERT only — never update old documents**

```
{ _id: ObjectId,
  userId: ObjectId,     // ref → users
  type: String,         // 'income' | 'expense' | 'transfer'
  amount: Number,
  category: String,     // 'Food' | 'Clothes' | 'Bonus' etc.
  account: String,      // 'Cash' | 'Bank'
  note: String,
  date: Date,           // STAR KEY FIELD — index this!
  createdAt: Date }
```

Why transactions is multiple per user:

```
// Same user abc123 — keeps growing forever
{ userId: 'abc123', amount: 200,  category: 'Food',    date: Mar 1  }
{ userId: 'abc123', amount: 500,  category: 'Clothes', date: Mar 5  }
{ userId: 'abc123', amount: 5000, category: 'Salary',  date: Mar 10 }
{ userId: 'abc123', amount: 100,  category: 'Travel',  date: Mar 15 }
```

> *IMPORTANT: The date field powers ALL weekly / monthly / yearly queries. This one field answers everything.*
>
> *Create index for fast queries: db.transactions.createIndex({ userId: 1, date: -1 })*
>
> *Every time user adds expense: INSERT in transactions + UPDATE balance in wallets. Both must happen together.*

D. goals

Purpose: Store how much the user wants to save each month.

Collection: goals

Documents per user: **1 per month — new document each month**

Operation: **INSERT for new month** | **UPDATE for same month**

```
{ _id: ObjectId,
  userId: ObjectId,        // ref → users
  targetAmount: Number,    // savings goal for this month
  month: Number,           // 1-12  (March = 3)
  year: Number,            // 2026
  createdAt: Date }
```

> *Check if goal exists for this month. If yes → UPDATE. If no → INSERT new document.*
> *When expense exceeds targetAmount → show confirmation warning to user.*

---

3. Documents Per User — Final Summary

users **always 1**
wallets **always 1**
transactions **unlimited — 1 per every expense or income**
goals **1 per month per user**

> *A user with 100 expenses has 100 transaction documents but still only 1 wallet document.*

---

4. Fetching Weekly / Monthly / Yearly Data

YES — all weekly, monthly, and yearly data is fetched from the transactions collection only. The date field makes this possible.

> *You NEVER create separate collections for weekly or monthly data. You NEVER store totals. MongoDB calculates everything using aggregate.*

How it works — same query, different dates

You write ONE function. You just pass different start and end dates depending on what the user selects.

Step 1 — Calculate dates in backend

```
const now = new Date();

// Weekly — last 7 days
const weekStart = new Date();
weekStart.setDate(now.getDate() - 7);
```

```
// Monthly — 1st of current month
const monthStart = new Date(now.getFullYear(), now.getMonth(), 1);

// Yearly — Jan 1st of current year
const yearStart = new Date(now.getFullYear(), 0, 1);
```

Step 2 — Run aggregate query

```
db.transactions.aggregate([
  {
    $match: {
      userId: ObjectId('abc123'),
      type: 'expense',
      date: { $gte: startDate, $lte: now }  // only change this
    }
  },
  {
    $group: {
      _id: '$category',           // group by Food, Clothes etc.
      total: { $sum: '$amount' }  // sum all amounts
    }
  },
  { $sort: { total: -1 } }        // highest first
])
```

> *$match = filter (which documents?). $group = calculate (sum, count). $sort = order (highest to lowest).*

Step 3 — What MongoDB does internally

```
// Example: monthly query finds these documents
{ amount: 200, category: 'Food',    date: Mar 5  }
{ amount: 500, category: 'Food',    date: Mar 12 }
{ amount: 300, category: 'Clothes', date: Mar 15 }
{ amount: 100, category: 'Travel',  date: Mar 20 }

// After $group and $sum:
{ _id: 'Food',    total: 700 }   // 200 + 500
{ _id: 'Clothes', total: 300 }
{ _id: 'Travel',  total: 100 }
```

For Bar Chart — Day by Day Breakdown

```
db.transactions.aggregate([
  { $match: { userId, type: 'expense', date: { $gte: weekStart, $lte: now } } },
  {
    $group: {
      _id: { $dayOfWeek: '$date' }, // 1=Sun 2=Mon ... 7=Sat
      total: { $sum: '$amount' }
    }
  },
  { $sort: { '_id': 1 } }
```

```
])
```

For Yearly — Month by Month Breakdown

```
db.transactions.aggregate([
  { $match: { userId, type: 'expense', date: { $gte: yearStart, $lte: now } } },
  {
    $group: {
      _id: { $month: '$date' },  // groups by month number 1-12
      total: { $sum: '$amount' }
    }
  },
  { $sort: { '_id': 1 } }
])
// result: [ { _id: 1, total: 5000 }, { _id: 2, total: 3200 } ... ]
//           Jan=5000       Feb=3200
```

Useful MongoDB Date Operators

$dayOfWeek → 1=Sun, 2=Mon ... 7=Sat — use for bar chart
$month → month number 1-12 — use for yearly breakdown
$year → year number — use for filtering by year
$week → week number of year — use for weekly grouping

> *You write the query ONCE. Only startDate changes based on what filter the user picks.*

---

5. Backend Route — One Route for All Filters

You do NOT write a separate query for weekly, monthly, and yearly. One route handles everything using query params.

Full Express route:

```
router.get('/expenses', async (req, res) => {

  const { filter } = req.query; // 'weekly' | 'monthly' | 'yearly'
  const now = new Date();
  let startDate;

  if (filter === 'weekly') {
    startDate = new Date();
    startDate.setDate(now.getDate() - 7);

  } else if (filter === 'monthly') {
    startDate = new Date(now.getFullYear(), now.getMonth(), 1);
```

```
  } else if (filter === 'yearly') {
    startDate = new Date(now.getFullYear(), 0, 1);
  }

  const result = await Transaction.aggregate([
    { $match: {
        userId: req.user._id,
        type: 'expense',
        date: { $gte: startDate, $lte: now }
    } },
    { $group: { _id: '$category', total: { $sum: '$amount' } } },
    { $sort: { total: -1 } }
  ]);

  res.json(result);
});
```

> *filter comes from req.query because it is a query param in the URL (?filter=weekly).*

---

6. All API Routes

Base URL: http://localhost:5000/api

Auth

```
POST   /api/auth/signup          ← register new user
POST   /api/auth/login           ← login with email + password
POST   /api/auth/verify-otp      ← verify OTP after signup
```

Transactions

```
POST   /api/transactions         ← add new expense or income
GET    /api/transactions?filter=weekly
GET    /api/transactions?filter=monthly
GET    /api/transactions?filter=yearly
GET    /api/transactions?filter=monthly&type=expense
```

Wallet

```
GET    /api/wallet               ← get current balance
```

Goals

```
POST   /api/goals                ← set monthly goal
GET    /api/goals                ← get current month goal
```

Report

```
GET    /api/report?filter=monthly ← download monthly report
GET    /api/report?filter=yearly  ← download yearly report
```

Profile

```
GET    /api/profile               ← get user info
PUT    /api/profile               ← update user info
```

---

7. Query Params — How They Work

Query params go after ? in the URL. They are used to send filters and options.

```
GET http://localhost:5000/api/transactions?filter=weekly
                                           ^ this is the query param


// Multiple params
GET /api/transactions?filter=monthly&type=expense
```

Access in Express:

```
const { filter } = req.query;   // 'weekly' | 'monthly' | 'yearly'
const { type }   = req.query;   // 'expense' | 'income'
```

Frontend sends like this:

```
// With axios
axios.get('/api/transactions', {
  params: { filter: 'weekly', type: 'expense' }
})
// axios adds ?filter=weekly&type=expense automatically
```

3 Ways to Send Data in Express

- **Query params** → ?filter=weekly → req.query.filter → use for filters and options
- **Route params** → /expenses/:id → req.params.id → use for specific item by ID
- **Body** → req.body → req.body.amount → use for POST data

---

8. Flow on Every Expense

Every time user adds an expense, these steps happen in order:

- User fills: date, amount, category, account, note
- Validate — check if balance is enough, else show error
- If expense exceeds goal → show confirmation warning
- User confirms with password
- INSERT new document in transactions collection
- UPDATE balance in wallets using $inc: { balance: -amount }
- Send email: 'You spent Rs X on Y. Your balance is now Rs Z.'

> *Both INSERT (transactions) and UPDATE (wallets) must always happen together in the same request.*

For income — reverse flow:

- INSERT new document in transactions with type: 'income'
- UPDATE balance in wallets using $inc: { balance: +amount }
- Send email notification

---

9. Key Rules to Remember

- **transactions** → INSERT only. Every expense or income = new document. Never update old ones.
- **wallets** → UPDATE only. Always 1 document per user. Use $inc — never $set.
- **users** → INSERT on signup. UPDATE on profile change. Always 1 per user.
- **goals** → Same month = UPDATE. New month = INSERT.

- **date field** → Index it! This is the key to all weekly/monthly/yearly queries.
- **Never store** → weekly totals or monthly totals. Always calculate using aggregate.
- **$inc** → Handles math automatically. No manual balance calculation needed.
- **One route** → Handles weekly/monthly/yearly. Only startDate changes dynamically.
- **Query params** → Use req.query.filter to read filter value in backend.

---

10. Next Steps — Start Coding

- Step 1 — Setup Express + MongoDB connection
- Step 2 — Create Mongoose models for all 4 collections
- Step 3 — Build auth routes (signup, login, OTP, Google OAuth)
- Step 4 — Build transaction routes (POST to add, GET with filter)
- Step 5 — Build wallet route (GET balance)
- Step 6 — Build goals route (set and check goal)
- Step 7 — Build report route (export data)
- Step 8 — Connect frontend to backend APIs
- Step 9 — Add email notifications

> *Schema is finalized and confirmed. You are ready to start building!*